Epistemic Typing as a PostgreSQL Table Access Method: Adversarial Conflict Resolution Under Confidence Forgery and Sybil Coordination

[Author Name Placeholder]
[Institution Placeholder]
[City Placeholder], [Country Placeholder]
[email@placeholder.example]

ABSTRACT

We describe KNDB, a PostgreSQL 18 table access method (TAM) that types every row with an engine-assigned epistemic kind (MEASURED, INFERRED, or DERIVED) and resolves per-slot conflicts inside every write-time heapam callback. Rows land as ordinary heap tuples; seven of the 42 TAM callbacks are overridden (tuple_insert, multi_insert, tuple_update, tuple_delete, tuple_insert_speculative, tuple_complete_speculative, relation_toast_am), the other 35 delegate to heap; we provide a completeness argument over the interface as a paper artefact. This paper reports the engineering behind that decision and the adversarial evaluation that motivated it. On a confidence-forgery workload where an attacker asserts INFERRED writes with confidence in $[0.95, 1.0]$ against honest MEASURED writes with confidence in $[0.5, 0.9]$, KNDB beats a confidence-only baseline by 63 percentage points on the Book-Author fusion dataset and 92.7 points on the Zheng crowdsourcing dataset. Both wins are proven load-bearing on the kind axis by a source-rebuild disable-and-test in which the lattice is neutralised and the win vanishes. Against four truth-discovery baselines (TruthFinder, CRH, CATD, ACCU) reimplemented from the original equations and validated to within 0.3 percentage points of the published numbers, KNDB is competitive below a per-dataset density-saturation cell and dominant at or above it. We formalise the cell as $k^* \approx \rho_{\text{alg}} \cdot h_{\text{top}}$, where h_{top} is per-slot top honest surface-form support, and validate the prediction within ±20% on Book-Author and ±30% on Zheng. Because the kind axis is assigned by the engine from independent metadata and cannot be forged at write time, KNDB's k^* is unbounded. The paper is honest about where KNDB loses: CRH and ACCU outperform KNDB below saturation on Zheng, and KNDB scores zero on three temporal knowledge-editing benchmarks whose ground truth is last-writer-wins.

1 INTRODUCTION

Databases now ingest writes from three broad classes of producer that disagree systematically about what they know. Instruments and user-facing forms produce claims that are directly observed. Machine-learning models and LLM agents produce claims that are inferred from a prompt or a feature vector. Rule engines produce claims that are derived from other rows already in the database. A conventional relational store treats these writes identically: whatever arrives last, or whatever a user-space BEFORE INSERT trigger accepts, becomes the current value. Under adversarial pressure this is unsafe. A miscalibrated LLM writer can flood a slot with high-confidence hallucinations that a confidence-ranking arbitrator will prefer over honest instrument readings. A coordinated set of Sybil agents can outvote honest sources on any slot where the Sybil count exceeds the top honest surface-form support.

KNDB is a PostgreSQL 18 extension that responds by lifting the epistemic kind of each write into the storage engine itself. Every row carries an ep_kind column (MEASURED, INFERRED, or DERIVED); at tuple_insert time a table access method (TAM) hook applies a total order on (kind, specificity, confidence, xmin) and evicts any live incumbent that loses. Because the hook runs inside heapam, no user-space bypass (DISABLE TRIGGER, session_replication_role = 'replica', COPY FROM) reaches around it. The engine is the enforcement point, not the trigger.

The central quantitative claim is Sybil-robustness at 1:1 density. Every truth-discovery (TD) baseline we tested collapses when the Sybil count k reaches or exceeds the per-slot top honest surface-form support h_{top}. KNDB does not, because kind rank is assigned by the engine from independent metadata that the adversarial identity does not possess (a source's n_listings history on Book-Author, a worker's quali_acc on a disjoint qualification test on Zheng). We prove this both empirically (Section 6) and as a threshold theorem (Section 5).

We are careful about scope. On workloads where kind and confidence coincide by construction, the lattice's kind axis buys nothing beyond confidence sorting, and KNDB matches pg_conf exactly (Section 6). On temporal knowledge-editing benchmarks whose ground truth is last-writer-wins, KNDB scores zero at $c = 1$, because its xmin tiebreak is first-committer-wins by design (Section 8). Below the density-saturation cell, CRH and ACCU outperform KNDB on the Zheng workload. These are stated up front and detailed in Sections 6 and 8.

Contributions.

(1) An engine-level epistemic type system implemented as a PostgreSQL 18 TAM. Three callbacks overridden (tuple_insert, multi_insert, relation_toast_am); the rest delegate to heap. Total 1789 lines: 1479 of C in src/ and 310 of headers in include/ (wc -l src/*.c include/*.h).
(2) An enumerated bypass model. We name six user-space write paths (DISABLE TRIGGER, session_replication_role, COPY FROM single-row and batch, INSERT, apply worker) and one that is out of scope (ALTER TABLE ...SET ACCESS METHOD heap). All six write paths are blocked by the TAM; the schema-change threat is disclosed. The batch COPY

path required a dedicated multi_insert override to close, described in Section 4.

(3) A confidence-forgery workload on two independent datasets, with source-rebuild disable-and-test in each case. Every quantitative claim in this paper corresponds to a JSON in bench/results/ whose dylib SHA-256 is recorded in the run metadata.

(4) A threshold theorem $k^* \approx \rho_{\text{alg}} \cdot h_{\text{top}}$ that predicts the Sybil-collapse cell of any dataset from its per-slot honest-support distribution alone. Empirically validated within ±20% on Book-Author (predicted bracket [3,5], observed cliff N=5…10) and ±30% on Zheng (predicted bracket [14,20], observed cell N=20).

(5) Honest limitations. TD baselines legitimately beat KNDB below saturation; KNDB scores 0.000 on three temporal benchmarks; there is a 15 000-row-per-transaction ceiling at PostgreSQL's default max_locks_per_transaction=64, inherited into COPY; ALTER TABLE …SET ACCESS METHOD heap sits outside the enforcement envelope; the custom WAL rmgr is an annotation channel, not a durability channel.

Source, benchmark harnesses, and every result JSON cited below are available at https://github.com/emailvenkatm/kndb/tree/postgres-experiment (branch postgres-experiment, tip 22d98ed at time of writing). Each result JSON in bench/results/ carries hardware and PostgreSQL configuration metadata, and each cell that involved a source rebuild records the dylib SHA-256 in its metadata block.

2 MOTIVATION AND THREAT MODEL

The threat model is stated up front and does not adapt to outcomes.

2.1 Confidence forgery

The confidence-forgery adversary asserts INFERRED writes with self-reported confidence uniform in $[0.95, 1.0]$ against a slot whose lattice-correct incumbent is a Tier-A MEASURED writer with self-reported confidence uniform in $[0.5, 0.9]$. Adversarial payload contains a scrambled real answer (an author string lifted from a different gold ISBN in Book-Author; a flipped binary label in Zheng). The real-world analogue is an LLM-generated claim that hallucinates a value but self-reports as certain. A confidence-only arbitrator ranks by confidence and prefers the adversary's higher self-report. KNDB is designed to reject the adversary regardless of the self-reported confidence, because MEASURED strictly outranks INFERRED in the lattice.

2.2 Sybil coordination

The Sybil adversary controls k synthetic identities per gold slot, each asserting the same falsified value. The adversary knows gold and picks the falsified value to maximise confusion (in binary tasks this is uniquely determined). The adversary cannot forge existing honest identities (Zheng worker IDs, Book-Author bookstore names) and cannot forge a MEASURED kind assignment. Kind assignment is engine-controlled and requires metadata that adversarial identities have no history of producing: on Book-Author, tier rank uses n_listings (source volume in book.txt) and canon_rate (fraction of author fields in "Last, First" format); on Zheng, tier rank uses quali_acc, computed from a disjoint qualification test with question IDs 2000…2019 that every worker took. Both signals live in the dataset before the conflict resolution runs, and both are unavailable to an adversary that appeared for the first time in the write stream. The independence self-audits are in the datasets' READMEs.

2.3 Write-path bypass

The engine-in-storage claim rides on the TAM callback being reachable from every user-space write path that a hostile writer with INSERT plus ALTER TABLE on the target relation can construct. Enumerated write paths, each verified against PostgreSQL 18 REL_18_STABLE source:

- ALTER TABLE …DISABLE TRIGGER ALL sets pg_trigger.tgenabled = 'D' (tablecmds.c:5588-5592); TriggerEnabled at trigger.c:3491-3499 returns false. A trigger-based enforcer is bypassable here; a TAM callback is not. Note that TriggerEnabled at trigger.c:3489-3499 still skips triggers marked TRIGGER_DISABLED even when session_replication_role = 'replica', so DISABLE TRIGGER ALL defeats ENABLE ALWAYS TRIGGER as well; the AM callback fires regardless of any pg_trigger.tgenabled value.
- SET session_replication_role = 'replica'. TriggerEnabled at trigger.c:3489-3499 skips TRIGGER_FIRES_ON_ORIGIN (the default for user CREATE TRIGGER) under SESSION_REPLICATION_ROLE_REPLICA. Triggers created ENABLE ALWAYS (tablecmds.c:5566, TRIGGER_FIRES_ALWAYS) are not in the disable-set at either branch of TriggerEnabled and DO fire under replica; that closes this specific bypass for a trigger-based enforcer that opts in to ENABLE ALWAYS. The GUC is PGC_SUSET (guc_tables.c:5166). The trigger-based enforcer therefore requires ENABLE ALWAYS to survive the replica bypass and still cannot survive DISABLE TRIGGER ALL; the AM callback survives both.
- COPY FROM single-row path. CopyFrom at copyfrom.c:995-1006 forces CIM_SINGLE when the target has a BEFORE trigger, an FDW that does not batch, or partitioned tables with statement-level triggers. The single-row path at copyfrom.c:1427 calls table_tuple_insert, which routes through the TAM.
- COPY FROM batch path (CIM_MULTI). The default path with no BEFORE trigger. Batched rows flow through table_multi_insert at copyfrom.c:554-559, dispatching on the AM's multi_insert. Before we overrode it (Section 4), this inherited heap_multi_insert and silently skipped every epistemic rule check.
- INSERT, INSERT …SELECT, INSERT …VALUES. Standard ModifyTable path lands in ExecInsert, which calls table_tuple_insert.

- UPDATE that rewrites the epistemic prefix (F20). ModifyTable on the CMD_UPDATE branch reaches table_tuple_update, which dispatches on the AM's tuple_update callback (tableam.h:718-727). Before F20 we inherited heapam_tuple_update (heapam_handler.c:392-400, delegating to heap_update at heapam.c:3241) and an UPDATE fact SET ep_kind='MEASURED', ep_confidence=1.0 against an incumbent INFERRED/0.4 row committed cleanly, forging the epistemic prefix on a live row without R1–R5 firing, without the advisory lock, and without the precedence lattice or eviction audit. The F20 override refuses any UPDATE whose new prefix differs from the incumbent's; content-only updates (value, valid_time, sources) are permitted.
- DELETE that removes an incumbent (F20). ModifyTable on CMD_DELETE reaches table_tuple_delete, which dispatches on the AM's tuple_delete callback (tableam.h:709-716). Before F20 we inherited heapam_tuple_delete (heapam_handler.c:382-388, delegating to heap_delete at heapam.c:2772). An adversary could DELETE a MEASURED incumbent and then INSERT an INFERRED forgery unopposed, since the precedence lattice has nothing left to compare against; no eviction audit row would ever be written for the MEASURED loss. The F20 override refuses DELETE outright with ERRCODE_FEATURE_NOT_SUPPORTED.
- INSERT ...ON CONFLICT (speculative-insertion protocol, F21). ExecInsert at nodeModifyTable.c:1189-1216 REL_18_STABLE handles ON CONFLICT by acquiring a SpeculativeInsertionLockAcquire, calling table_tuple_insert_speculative (tableam.h:513-519) to write the tuple with a speculative token, probing arbiter indexes via ExecInsertIndexTuples, then calling table_tuple_complete_speculative (tableam.h:521-525) to either confirm (heap_finish_speculative) or kill (heap_abort_speculative) the tuple. Before F21 the AM inherited heapam_tuple_insert_speculative and heapam_tuple_complete_speculative verbatim (heapam_handler.c:2639-2640); an INSERT ...ON CONFLICT DO UPDATE or DO NOTHING that reached the speculative path would bypass R1–R5, the advisory lock, precedence, and the eviction audit. *The SQL surface for this attack is currently unreachable on epistemic tables* because CREATE UNIQUE INDEX, ADD PRIMARY KEY, ADD UNIQUE, and ADD EXCLUDE all fail at heap_getnext's rd_tableam identity check (heapam.c:1352) during the arbiter index's build scan, so ON CONFLICT (col) has no arbiter to bind (see scripts/speculative_forgery.sh for the empirical SQL-level probe). The F21 overrides are defensive: the engine-in-storage claim must not depend on that accidental index-support gap persisting. The load-bearing proof lives in a C-level probe (epistemic._probe_speculative_insert) and its sql/am_speculative.sql regression cell, exercised by the source-rebuild disable-and-test in Section 6.6.
- Logical replication apply worker. Runs as SESSION_REPLICATION_ROLE_REPLICA by default; still reaches the TAM via ExecSimpleRelationInsert → ExecInsert → table_tuple_insert. Enforcement fires.
- ALTER TABLE ...SET ACCESS METHOD heap. Out of scope. This is a schema-change threat available to the table owner only; it rewrites the relation onto plain heap and removes the TAM from the write path entirely. See Section 8.

The engine-in-storage differentiator holds against a writer with INSERT, UPDATE, DELETE, plus ALTER TABLE on the target relation, or a role that can toggle session_replication_role (a replication or CDC operator, a migration tool, a connection pool). It does not hold against the table owner.

3 DESIGN

KNDB adds one PostgreSQL access method (CREATE ACCESS METHOD epistemic USING TABLE HANDLER epistemic_am_handler) and one base type (epistemic_kind, a pass-by-value byte). Tables declared USING epistemic carry three extra columns beyond the user schema (ep_kind, ep_specificity, ep_confidence) and a bitemporal sys_time tstzrange. Rows are stored as ordinary heap tuples; storage format on disk is identical to a heap table with the same schema.

3.1 Precedence lattice

At write time a candidate row and any live incumbent for the same (entity_id, attribute) slot are ranked by the total order:

$$\underbrace{\text{kind}}_{\text{MEASURED}>\text{DERIVED}>\text{INFERRED}} \succ \text{specificity} \succ \text{confidence} \succ \text{xmin}$$

Ranks are integer-compared at each level. Kind rank is the primary sort key. The lattice is defined in epistemic_rules.c and exposed as an idempotent SQL function tested by sql/precedence.sql.

3.2 Write-time rules R1–R5

The TAM enforces five predicates on the candidate before ranking, checked in order at the top of epistemic_tuple_insert_impl; the first failure ereports ERRCODE_CHECK_VIOLATION and aborts the insert. Reference implementation: contrib/epistemic/src/epistemic_rules.c.

- R1 (DERIVED sources): if the candidate's kind is DERIVED, its sources[] column must be non-empty. A DERIVED assertion without a source is refused.
- R2 (source resolution): if the candidate's kind is non-MEASURED and sources[] is non-empty, every source identifier must resolve against epistemic.source_registry. R2 uses SPI to look up each source id and refuses the write if any is unregistered. This is the registry that prevents an adversary from inventing new source identities on the fly.
- R3 (MEASURED no sources): if the candidate's kind is MEASURED, sources[] must be empty. A MEASURED row is a first-hand observation and does not derive from other sources.

- R4 (INFERRED confidence bound): if the candidate's kind is INFERRED, ep_confidence must be strictly less than 1.0. An INFERRED row that claims certainty is refused; certainty is the property MEASURED asserts.
- R5 (slot kind registry): if epistemic.slot_kind has a row for the candidate's attribute, the candidate's kind must match required_kind. R5 lets an operator pin an attribute to a specific kind (e.g., "a hemoglobin_A1c reading must be MEASURED, never INFERRED").

R1 through R5 are content-level constraints on individual candidate rows and are independent of the precedence lattice; a candidate that passes R1–R5 still competes against any live incumbent through the precedence check that follows. The confidence-forgery attack we exercise in Section **??** uses INFERRED writes with ep_confidence in $[0.95, 1.0)$: adversarial writes are chosen precisely so they pass R4 (and R1, R2, R3, R5), forcing the kind axis of the precedence lattice to be the mechanism that defeats them.

3.3 Per-slot advisory lock (F6)

Between the R1–R5 check and the incumbent scan, the TAM takes a LOCKTAG_ADVISORY transaction-scope lock with (field2=entity_id, field3=hash_bytes(attribute), field4=2), constructed inline via SET_LOCKTAG_ADVISORY and acquired with LockAcquire(&tag, ExclusiveLock, false, false). This is the same tag, mode, and scope as pg_advisory_xact_lock_int4. It serialises concurrent writers on their shared (entity_id, hashed attribute) key, so the incumbent scan runs against a snapshot that has already absorbed any peer's just-committed row. find_live_overlap and epistemic_close_sys_time both use GetLatestSnapshot() for the same reason.

Without this lock, two concurrent same-slot writers each see an empty slot in their own MVCC snapshot, and both commit. The rc_invariant.sh script runs 50 trials of two overlapping same-slot inserts. Under the honest build, session1 wins all 50 and no both-live rows land. With the advisory-lock block patched to if (0) and the dylib rebuilt, both writers commit in 50 of 50 trials.

3.4 First-committer-wins tiebreak (F8)

On a true (kind, specificity, confidence) tie the TAM decides by reading the incumbent's raw xmin via HeapTupleHeaderGetRawXmin and comparing against the current backend's xid via TransactionIdPrecedes (which handles xid wraparound). Under the advisory lock, the incumbent's transaction has already committed by the time the loser scans it, so incumbent_xmin $<$ new_xid on every race. The incumbent wins.

The earlier design used a content-hash tiebreak. That gave up grind-resistance in exchange for content-determinism: an attacker with SELECT plus INSERT could iterate byte-level variations of value until hash_bytes placed the attacker below the incumbent, and scripts/hash_grind.sh at F7 reported 30 of 30 attacker wins with a mean of 7.5 attempts (min 1, max 95). Under the xmin tiebreak the same script reports 0 of 20 000 attacker wins across 100 trials of 200 attempts. The tradeoff we accepted: survivor is start-order-dependent and is not stable across pg_dump/pg_restore, because restore reloads rows via COPY FROM at copyfrom.c:1427 and each reloaded row gets a fresh xid. What is stable across dump and restore is the "exactly one live row per slot" invariant that sql/am_eviction.sql asserts.

3.5 Reason codes and audit

On eviction the TAM writes one audit row into epistemic.evicted_fact via SPI and closes the incumbent's sys_time upper bound via simple_heap_update. The audit row carries the losing row's identifiers and a reason code from EP_REASON_{OUTRANKED_KIND, OUTRANKED_SPEC, OUTRANKED_CONF, CONTRADICTED_SAME_RANK}. All three state changes (winner insert, audit insert, incumbent update) execute inside one top-level PostgreSQL transaction created by start_xact_command; the TAM does not open, commit, or manage transactions. Atomicity is inherited from PostgreSQL. An adversarial control that stages the audit row through dblink (which commits in a separate backend) produced 1–2 torn-state violations per 25 crash trials at crash_atomicity_broken.sh; the honest in-transaction path produced 0 of 25.

4 IMPLEMENTATION

KNDB is a shared_preload_libraries extension of 2274 lines of C in src/ plus 310 lines of headers in include/ (2584 total per wc -l, of which 183 are the F21 C-level test probe src/epistemic_probe.c, leaving 2401 lines in the production surface). It targets PostgreSQL 18 REL_18_STABLE via PGXS. Building the extension against a stock 18.4 install requires no PostgreSQL patch.

4.1 Handler and delegation

The AM handler copies the heap AM's TableAmRoutine at first call and overrides seven entries:

- tuple_insert (single-row insert path from ExecInsert);
- multi_insert (batch path from COPY FROM CIM_MULTI, added at F18);
- tuple_update (single-row update path from ExecUpdate, added at F20; rejects any UPDATE that alters ep_kind, ep_specificity, or ep_confidence);
- tuple_delete (single-row delete path from ExecDelete, added at F20; rejects DELETE outright);
- tuple_insert_speculative (first phase of INSERT ...ON CONFLICT from ExecInsert, added at F21; runs R1–R5, the advisory lock, and precedence; delegates the speculative write to heap; stashes any pending eviction until tuple_complete_speculative);
- tuple_complete_speculative (second phase, added at F21; on succeeded=true drains the pending eviction, on succeeded=false discards it so no audit row is written for a killed speculative winner);
- relation_toast_am (delegates TOAST to the same AM so long values stay under the epistemic table's namespace).

The remaining ~35 callbacks (scan_begin, scan_getnextslot, tuple_lock, index_fetch_*, relation_set_new_filelocator, relation_copy_data, relation_vacuum, and so on) are heap's, unmodified. Reads, index builds, VACUUM, and analyse all inherit heap's behaviour byte-for-byte. Section 4.9 converts this "~35"

claim into an exhaustive audit (Table 1) with a citable reason per row.

4.2 The tuple_insert wrapper

On each call, epistemic_tuple_insert_impl runs the following sequence:

(1) R1–R5 rule check on the candidate row.
(2) Per-slot advisory transaction lock (Section 3).
(3) find_live_overlap scan against GetLatestSnapshot() for a live row in the same (entity_id, attribute) slot.
(4) epistemic_precedence_cmp between candidate and incumbent, including the xmin tiebreak.
(5) If NEW_LOSES, raise epistemic precedence: NEW_LOSES (reason=...) and the transaction rolls back.
(6) If NEW_WINS, call heapam.tuple_insert for the winner, epistemic_close_sys_time for the incumbent, and epistemic_audit_evicted for the audit row.
(7) Emit one annotation record on custom WAL rmgr 128 (epistemic_wal_log_insert_marker).

Step 7 is not load-bearing for durability. Section 4.6 explains.

4.3 The multi_insert wrapper (COPY FROM)

PostgreSQL 18's CopyFrom at copyfrom.c:995-1006 selects insertMethod=CIM_MULTI when the target has no BEFORE or INSTEAD OF INSERT trigger, then buffers up to 1000-tuple batches (MAX_BUFFERED_TUPLES) and flushes them via table_multi_insert at copyfrom.c:554-559. Before F18 we copied heap's TableAmRoutine and inherited heap_multi_insert unchanged, which meant that every COPY-batched row skipped R1–R5, the advisory lock, precedence, eviction, and the audit. The bad row landed silently; the transcript reads COPY 1 or COPY 5 with no error.

F18 overrides multi_insert with a for-loop that invokes epistemic_tuple_insert_impl(rel, slots[i], cid, options, bistate) for each slot in the batch. This is byte-for-byte identical enforcement to a single-row INSERT of the same rows. Correctness is preserved over throughput: heap_multi_insert would toast in bulk, pack tuples onto pages with amortised allocation, and emit one XLOG_HEAP2_MULTI_INSERT WAL record per page; the per-slot fan-out pays one heap_insert per row and one WAL record per row. COPY into an epistemic table therefore runs at roughly the throughput of INSERT ...SELECT, not of a plain-heap COPY.

We chose full override over loud rejection because pg_dump | pg_restore regenerates \copy statements and refusing them would break restore for any epistemic table. The alternative to per-slot advisory locks in the batch path would have been to drop the lock; that silently re-opens the RC integrity leak that F6 closed.

4.4 The tuple_update and tuple_delete wrappers (F20)

Before F20 the AM inherited heapam_tuple_update and heapam_tuple_delete verbatim (heapam_handler.c:1384-1385 REL_18_STABLE bindings; heap_update at heapam.c:3241, heap_delete at heapam.c:2772), so an adversary with INSERT plus UPDATE could overwrite the epistemic prefix on a live row without any rule check, without the advisory lock, without the precedence lattice, and without an eviction audit row — and an adversary with DELETE could remove a MEASURED incumbent so that a subsequent INSERT landed unopposed. The attack transcripts live in scripts/update_forgery.sh and scripts/delete_forgery.sh.

The naive fix — re-run the tuple_insert enforcement path on the candidate slot with the incumbent-as-self supplied to the precedence lattice — fails: a NEW MEASURED/1.0 row legitimately beats an OLD INFERRED/0.4 row by kind rank, so the attack succeeds through the lattice. The only defensible cut is at the prefix. epistemic_tuple_update fetches the incumbent tuple at otid via heap_fetch(rel, GetLatestSnapshot(), ...), deforms it, compares (ep_kind, ep_specificity, ep_confidence) against the candidate slot, and ereport(ERRCODE_CHECK_VIOLATION)s if any of the three differ. Legitimate content updates (value, valid_time, sources) reach heapam_tuple_update unchanged, so pt-osm-style column edits still work.

epistemic_tuple_delete refuses DELETE outright with ERRCODE_FEATURE_NOT_SUPPORTED. The stricter option — allow DELETE only for rows whose sys_time upper bound is closed, and always write an audit row — would layer atop this and is left to a future eviction API. The rationale for cutting hardest at delete: any DELETE of a live row corresponds to an eviction event that the precedence lattice should have arbitrated, and there is no adversary-friendly workflow that requires the caller to bypass that arbitration.

4.5 The speculative-insertion wrappers (F21)

INSERT ...ON CONFLICT routes through a two-phase protocol at nodeModifyTable.c:1189-1216 REL_18_STABLE: table_tuple_insert_speculative writes the tuple with a speculative token, ExecInsertIndexTuples probes the arbiter index for a conflict, and table_tuple_complete_speculative either confirms or kills the tuple. Before F21 we inherited heapam_tuple_insert_speculative and heapam_tuple_complete_speculative verbatim, so an ON CONFLICT that reached the speculative path would skip every enforcement step. As Section 2.3 records, that path is not currently *reachable* from SQL because unique/exclusion index creation fails on epistemic tables at heap_getnext's rd_tableam identity check (heapam.c:1352), so ON CONFLICT (col) has no arbiter to bind. The overrides are defensive; the engine-in-storage claim must not depend on the accidental index-support gap persisting, and the C-level probe at sql/am_speculative.sql proves they are load-bearing.

epistemic_tuple_insert_speculative runs the same first four steps as the plain insert path: R1–R5, the F6 advisory xact lock, find_live_overlap, precedence with the F8 xmin tiebreak. The speculative write itself is delegated to heapam->tuple_insert_speculative so heap stamps the tuple with HEAP_INSERT_SPECULATIVE and the caller-supplied specToken. Eviction bookkeeping (audit row +

sys_time close + rmgr-128 marker) is deferred: if the arbiter check at nodeModifyTable.c:1199 concludes a conflict, table_tuple_complete_speculative(succeeded=false) calls heap_abort_speculative (heapam.c:6186) and the winner tuple vanishes. Writing the audit row before the confirm phase would leave the store with an evicted incumbent and no winner — a durability violation strictly worse than the bypass.

epistemic_tuple_complete_speculative therefore drains a single-entry backend-local pending-eviction slot keyed by specToken. On succeeded=true the slot's contents are folded into epistemic_audit_evicted + epistemic_close_sys_time + epistemic_wal_log_insert_marker, mirroring the plain-insert bookkeeping. On succeeded=false the slot is discarded. The F6 advisory lock is xact-scope and stays held across both callbacks; it is released at outer-transaction commit/abort, not at speculative-complete, so a peer that raced us waits until we finish. The single-entry assumption is safe because ExecInsert at nodeModifyTable.c:1189-1216 holds SpeculativeInsertionLockAcquire across both calls, so a backend performs at most one speculative insertion at a time.

4.6 WAL: annotation, not durability

The extension registers custom resource manager id 128 (RM_EXPERIMENTAL_ID) and emits one XLOG_EPISTEMIC_INSERT record per row after the heap insert commits. This record is intentionally minimal: tuple_len=0, no buffer reference. Heap's XLOG_HEAP_INSERT at heapam.c:2222-2226 already carries every column of the tuple, including ep_kind, ep_specificity, ep_confidence, and sys_time, and heap_xlog_insert at heapam_xlog.c:482-503 reconstructs the row at redo. We verified empirically (scripts/recovery.sh, 110 rows round-trip; wal_consistency_checking=all) that recovery succeeds byte-for-byte with the rmgr-128 emitter disabled. The custom rmgr is retained as a named channel a future logical decoding consumer can hook into; it is not load-bearing for durability of the row.

4.7 Bulk-load ceiling

The advisory lock is per-row and lives in PostgreSQL's per-transaction fast-path lock table (lock.c:56-57, NLOCKENTS = max_locks_per_xact × (MaxBackends + max_prepared_xacts)). At PostgreSQL's default max_locks_per_transaction=64, one transaction inserting ~15 000 distinct-slot rows exhausts the shared lock table and raises ERROR 53200: out of shared memory with the hint to raise max_locks_per_transaction. scripts/lock_exhaustion_linearity.sh sweeps the GUC and shows the ceiling scales linearly (raising to 1024 pushes it to ~250 000). The ceiling applies to COPY too, in the same shape: scripts/bypass.sh sweeps $N \in \{100, 1000, 10000, 20000\}$ via real \copy at the default GUC and the first three succeed while $N = 20000$ raises ERROR 53200 at LockAcquireExtended, lock.c:1080. This is an accepted design tradeoff: dropping the lock in the batch path would re-open the F6 integrity leak; raising the GUC (a one-line change in postgresql.conf) removes the ceiling in a way operators can already control.

4.8 Threat surface, restated

Every callback listed above runs from inside heapam. No user-space GUC or ALTER TABLE reaches it. The one exception is ALTER TABLE ...SET ACCESS METHOD heap, which is a schema-change threat available to the table owner only; it rewrites the relation onto plain heap, at which point the callback is out of the write path entirely, because the callback is no longer bound to the relation. This is out of scope and disclosed in Section 8.

4.9 Exhaustive TAM audit

Enumerating write paths anecdotally has been wrong three times in this work: F9 found the COPY CIM_MULTI bypass, F20 found the UPDATE/DELETE bypass, and F21 found the speculative-insertion bypass. Each was a callback we had classified as "heap's, unmodified, cannot mutate the epistemic invariant" and each turned out to route a real write. Table 1 converts anecdotal enumeration into a completeness argument: every one of the 42 callbacks in the PG 18 REL_18_STABLE TableAmRoutine interface (access/tableam.h) is classified with a citable file:line reference and a stated reason why it is either overridden or safe to inherit. The classification method is: (i) read the callback signature and docstring; (ii) find heap's binding in heapam_handler.c and read the body; (iii) grep for every caller in src/backend and confirm the caller either never mutates epistemic-visible state or routes back through a callback we do override. Seven callbacks are *overridden* (each with its own source-rebuild disable-and-test in scripts/ and each with the dylib hash flip recorded in DECISIONS.md); the other 35 are inherited. Two rows are flagged *delegated-storage* with the honest disclosure that TRUNCATE and CLUSTER/VACUUM FULL do bypass epistemic policy: both are DDL-privileged, not write-path, so they belong alongside SET ACCESS METHOD heap in the schema-change threat category disclosed in Section 8.

5 FORMALISM

The Sybil-collapse threshold of an agreement-based truth-discovery algorithm is predictable from the dataset's per-slot honest-support distribution alone. This section makes that statement precise and validates it empirically on the two datasets in the evaluation. The complete derivation is in the companion bench/docs/sybil_formalism.md; this section reports what the paper needs.

5.1 Model and threat

Let $S = S_h \cup S_s$ be the disjoint sets of honest and Sybil sources, I the items, $v^*(i)$ the true value at item i. A claim is a triple (i, s, v). Define

$$h_{\text{top}}(i) = \max_{v \in V_i^*} |\{s \in S_h : (i, s, v) \text{ claimed}\}|,$$

where V_i^* is the set of value strings the correctness scorer accepts as matching gold. For binary tasks (Zheng d_sentiment) $V_i^* = \{v^*(i)\}$ and $h_{\text{top}}(i) = h(i)$. For string-valued tasks (Book-Author) gold-matching claims are split across surface forms ("O'Leary, Timothy J." versus "Timothy J O'Leary"), so $h_{\text{top}}(i) \le h(i)$.

Each TD algorithm assigns each source a weight $w(s) \geq 0$ and picks $v(i) = \arg\max_v \sum_{s:c(s,i)=v} w(s)$ with algorithm-specific tie-break. A Sybil coalition of size k per slot asserts one falsified value $f(i) \neq v^*(i)$.

5.2 Per-algorithm monotonicity

The four algorithms in the evaluation are TruthFinder [12], CRH [4], CATD [3], and ACCU [2]. For each, the summed weight of k Sybils asserting one value grows monotonically in k: TruthFinder's fixed-point iteration bootstraps supporter trust from mutual agreement (KDD 2007 eqs. 3, 7, 8); CRH's log-ratio yields a defensive $O(1/\ln|I|)$ margin against a single Sybil but not against a coalition; CATD's $\chi^2_{\alpha/2,n}$ upper confidence bound tightens as n grows but the summed weight is linear in k; ACCU's Bayesian MAP over per-source accuracy has a per-supporter log-odds coefficient that stays positive for any $A > 1/(1 + n_{\text{false}})$. Details in the companion document.

5.3 Threshold theorem

Under standard hyperparameters (TruthFinder: $\gamma = 0.3$, $\rho = 0.5$, initial trust 0.9; CRH: default; CATD: $\alpha = 0.05$; ACCU: initial $A = 0.8$), for each of the four algorithms there exists a finite threshold

$$k^*(i) \lesssim \rho_{\text{alg}} \cdot h_{\text{top}}(i)$$

above which the Sybil coalition flips $v(i)$ from $v^*(i)$. The per-algorithm constants ρ are bounded above by $1 + o(1)$: $\rho_{\text{TF}} \approx 1$ (sometimes below 1 in the sub-saturation regime because the iteration is self-amplifying); $\rho_{\text{CRH}} \approx 1+1/\ln|I|$; $\rho_{\text{CATD}} \approx 1$ at $\alpha = 0.05$; $\rho_{\text{ACCU}} \approx 1$ on binary tasks. The consequence is $k^* = \Theta(h_{\text{top}})$: the Sybil-collapse cell is linear in per-slot honest support with a per-algorithm slope close to one.

5.4 KNDB invariance

Under the F1–F8 KNDB lattice with tier mapping computed from independent metadata, k^* is unbounded. Proof: the lattice orders kind strictly above every other axis, and kind $\in$ {MEASURED, INFERRED, DERIVED} is assigned at write time by a rule that consults only metadata living in the dataset before conflict resolution. On Book-Author, Tier A (MEASURED) requires membership in the top $K/2$ by n_listings *and* canon_rate ≥ 0.5; an adversarial identity appearing for the first time in the trace has n_listings $\leq k \ll K/2$ and no canon_rate history. On Zheng, Tier A requires top-third quali_acc on the disjoint qualification test (question IDs 2000–2019); an adversarial identity has no qualification responses. In both cases the adversary is at most INFERRED. MEASURED strictly outranks INFERRED, so a single Tier-A MEASURED honest write beats any coalition of INFERRED writes regardless of coalition size or confidence values. Therefore $k^*(i) = \infty$.

The empirical counterpart of this theorem is the F14/F15b KIND_OFF disable-and-test (Section 6): when the kind-primary ordering is patched out and only confidence remains, KNDB collapses to pg_conf's numeric behaviour on both datasets.

5.5 Empirical validation

We measured h_{top} directly on both datasets. Book-Author K=50: median $h_{\text{top}} = 4$, mean 7.3, p90 18. Predicted collapse bracket $k^* \in [3, 5]$; observed sharp cliff at N=5…10 for TruthFinder, CATD, and ACCU (TruthFinder $0.530 \rightarrow 0.120$ at N=5, $\rightarrow 0.010$ at N=10; CATD $0.550 \rightarrow 0.420 \rightarrow 0.100$; ACCU $0.530 \rightarrow 0.290 \rightarrow 0.060$); CRH lags one step (0.580 at N=5, 0.230 at N=10). The predicted bracket is within ±20% of the observed cliff.

Zheng d_sentiment K=45: median $h_{\text{top}} = 14$, uniform 20 votes per slot. Predicted collapse bracket $k^* \in [14, 20]$; observed collapse at N=20 with CRH, CATD, and ACCU all going to zero. Within ±30% of the observed threshold on the k-axis; the observed cell is the density-saturation cell exactly.

6 EVALUATION

6.1 Experimental setup

All measurements are on PostgreSQL 18.4 at /tmp/kndb_pg18_test:55480, one Apple M5 Pro (18 cores, 48 GB RAM, macOS 26.4.1), unix-socket connections. The extension is loaded via shared_preload_libraries = 'epistemic' and defaults otherwise, except where a specific cell required raising max_locks_per_transaction for a bulk-load probe. Every bench cell records the extension dylib SHA-256 in its metadata block. The reference honest build in this paper is dylib SHA-256 eb15d442dd1eace588c1c4ee4fab3e183addc70cc3a28773-f8d5acfc0ff58af0, the F18 post-COPY-close artefact. Cells taken before F18 are on 807b2e87..., disclosed inline where relevant. Every source-rebuild disable-and-test recorded both the honest and patched dylib SHA-256 and confirmed the source tree returned byte-identical to HEAD after the run.

6.2 Confidence-forgery workload (Book-Author)

Dataset: Dong et al. VLDB 2009 Book-Author fusion [2], 895 bookstores, 1265 books, 33 971 assertions, gold for 100 ISBNs. Tier map (independence audit in bench/datasets/bookauthor/README.md): Tier A (MEASURED, conf uniform $[0.5, 0.9]$) if top-$K/2$ by n_listings *and* canon_rate ≥ 0.5; Tier B (INFERRED, conf $[0.4, 0.7]$); Tier C (DERIVED, conf $[0.2, 0.5]$). Adversarial injection (N per gold ISBN) is INFERRED with conf $[0.95, 1.0]$ carrying a scrambled real author name from a different gold ISBN. K=50 held fixed; N swept over $\{1, 3, 5, 10\}$; seed 20260714 fixes RNG.

Table ?? reports precision on gold ISBNs at $c = 1$. KNDB stays at 0.630 across every N (kind rank picks Tier-A MEASURED over adversarial INFERRED); pg_conf collapses to 0.000 at every N (ranked by confidence alone, the adversarial $[0.95, 1.0]$ beats Tier-A $[0.5, 0.9]$); pg_lww degrades $0.210 \rightarrow 0.040$; pg_mv $0.460 \rightarrow 0.170$; pg_heap is an integrity failure (max 124 live rows per slot, mean 38.6) and its numeric precision (0.68–0.71) is a coin flip over which duplicate the scan returned first. pg_trigger tracks KNDB to within one point (same lattice, plpgsql implementation, marginally higher abort rate under contention).

6.3 Confidence-forgery workload (Zheng)

Dataset: Zheng et al. VLDB 2017 d_sentiment crowdsourcing task [13], 85 AMT workers, 1000 items, ~20 labels per item, 999 of 1000 items contested. Tier map (independence audit in bench/datasets/zheng_sentiment/README.md): rank workers by quali_acc(w) on the disjoint qualification test (item IDs 2000–2019, 1700 responses); Tier A (MEASURED) = top $K/3$; Tier B (INFERRED); Tier C (DERIVED, all workers outside top-K). Adversarial injection (N per contested slot) is INFERRED with conf $[0.95, 1.0]$ and value = binary flip of gold. K=45 held fixed. Table **??** reports precision at $c = 1$.

The delta over pg_conf is 92.7 points, flat across N. The delta over KNDB's second-best PostgreSQL baseline (pg_lww) is 52–85 points depending on N.

6.4 Source-rebuild disable-and-test

On both datasets we patched epistemic_precedence_cmp (src/epistemic_rules.c:379-423) to force inc_rank = new_rank = 1, short-circuiting the kind branch so the function falls through to specificity and confidence. This is the exact ranking behaviour of pg_conf on both workloads (specificity is 0 across the board; confidence decides). After each measurement the source was restored byte-identical (git diff –stat contrib/epistemic/src/ empty), rebuilt and reinstalled, and the dylib hash returned to the honest value.

Book-Author, N=5, $c = 1$: KNDB kind ON precision 0.630, kind OFF precision 0.000 (dylib flip 807b2e87... → f7... → 807b2e87...). Delta −63 points, exactly the KNDB-versus-pg_conf margin. Zheng, N=5, $c = 1$: KNDB kind ON precision 0.927, kind OFF precision 0.000 (dylib flip 807b2e87... → 3cc4f4b7... → 807b2e87...). Delta −92.7 points, exactly the KNDB-versus-pg_conf margin.

Integrity held under the patched build in both cases (no slots with more than one live row) because the F6 advisory lock and F8 xmin tiebreak still operated; only kind-rank decision-making was disabled. This is the correct decomposition: integrity and correctness are separable mechanisms in KNDB, and each has its own disable-and-test.

6.5 UPDATE and DELETE forgery attacks (F20)

Two attacks the earlier bypass-table enumeration missed: an UPDATE that rewrites the epistemic prefix on a committed row, and a DELETE that removes an incumbent so that a subsequent INSERT lands unopposed. Both were reproduced against a fresh cluster before writing the fix: scripts/update_forgery.sh seeds an INFERRED/0.4 row, issues UPDATE fact SET ep_kind='MEASURED', ep_confidence=1.0, value='forged', and reads back MEASURED/1.0/forged without any error, without any eviction audit row. scripts/delete_forgery.sh seeds a MEASURED incumbent, deletes it, inserts an INFERRED/0.99 row, and reads back INFERRED/0.99/benign as the live row for the slot.

F20 adds two callbacks. epistemic_tuple_update fetches the incumbent at otid via heap_fetch with GetLatestSnapshot, deforms it, and refuses any UPDATE whose new (ep_kind, ep_specificity, ep_confidence) triple differs from the incumbent's, raising ERRCODE_CHECK_VIOLATION. epistemic_tuple_delete refuses DELETE outright with ERRCODE_FEATURE_NOT_SUPPORTED. Content-only UPDATEs that change value, valid_time, or sources delegate to heapam_tuple_update unchanged, so a legitimate content-correction workflow still succeeds. The regression test cell sql/am_update_delete.sql exercises both callbacks under make installcheck.

The disable-and-test proof: with the F20 wiring in epistemic_am_handler commented out and the extension rebuilt (dylib flip c873ddc4... → 1fb0d572...), both forgery scripts print FORGERY SUCCEEDED; restoring the two wiring lines byte-identical and rebuilding (1fb0d572... → c873ddc4...) returns both to FORGERY REJECTED. This confirms the wiring is load-bearing and neither callback is dead code, mirroring the F18 discipline for multi_insert. Under the honest build, the extension's make check-e2e suite now passes 10/10 (adding check-e2e-updforge and check-e2e-delforge to the F19 suite of 8), and scripts/bypass.sh passes 28 assertions covering five bypass mechanisms × two tables plus the F7 lock-table sweep.

6.6 Speculative-insertion attack (F21)

F21 adds two callbacks. epistemic_tuple_insert_speculative runs R1–R5, the F6 advisory xact lock, and precedence (with the F8 xmin tiebreak), then delegates the actual write to heap's heapam_tuple_insert_speculative so the row is stamped with HEAP_INSERT_SPECULATIVE and the caller's specToken. Eviction bookkeeping is deferred to epistemic_tuple_complete_speculative: on succeeded=true the callback drains a backend-local pending-eviction slot (audit row + sys_time close + rmgr-128 marker); on succeeded=false it discards the slot, because heap_abort_speculative has just removed the winner tuple. The two-phase deferral is load-bearing: writing the audit row before the confirm phase would leave the store with an evicted incumbent and no winner if the arbiter check aborted the speculative row — worse than the bypass we are closing.

The SQL-level attack surface for the speculative path is not currently reachable on epistemic tables. CREATE UNIQUE INDEX, ADD PRIMARY KEY, ADD UNIQUE, and ADD EXCLUDE all fail on epistemic relations at heap_getnext's rd_tableam identity check (heapam.c:1352 REL_18_STABLE) during the btree/gist ambuild scan, so ON CONFLICT (col) has no arbiter to bind. scripts/speculative_forgery.sh confirms empirically that four SQL-level attempts (DO UPDATE with a UNIQUE arbiter that never gets built, DO NOTHING with a target column list, bare DO NOTHING, and constraint-creation probes) all either fail at DDL or fall back to a route already covered by the plain-tuple_insert override. That is a happy accident of the same index-support gap that drives the ~71% overhead in Section 6; the engine-in-storage claim must not depend on it.

The C-level probe epistemic._probe_speculative_insert, added at F21 alongside the two overrides, invokes table_tuple_insert_speculative and table_tuple_complete_speculative programmatically with a caller-supplied candidate tuple, mirroring ExecInsert's two-phase call sequence at nodeModifyTable.c:1189-1216. The regression

cell sql/am_speculative.sql exercises the probe against seven distinct cases: valid MEASURED, R3 violation, valid INFERRED, R4 violation, precedence NEW_LOSES, precedence NEW_WINS with deferred eviction, and speculative-abort with a would-be eviction that must be discarded. The disable-and-test proof: with the F21 wiring in epistemic_am_handler commented out and the extension rebuilt (dylib flip 959d5e67... → c44b276d...), the probe with an R3-violating MEASURED row and the probe with an R4-violating INFERRED row both return OK and the forged row lands in the target relation; restoring the two wiring lines byte-identical and rebuilding (c44b276d... → 959d5e67...) returns both to the expected ERRCODE_CHECK_VIOLATION. Under the honest build, make installcheck passes 8/8 (adding am_speculative to the F20 suite of 7).

6.7 Truth-discovery baselines

We reimplemented TruthFinder, CRH, CATD, and ACCU in Python 3 from the original equations (bench/scripts_td/td_algorithms.py). No vendored code. Each algorithm reproduced the Zheng VLDB 2017 survey's D_PosSent baseline to within 0.3 percentage points (CATD 0.957 vs. survey 0.960; CRH 0.950 vs. survey PM 0.9504). Default hyperparameters per each paper; no tuning either way. TD algorithms consume the same normalised trace KNDB and pg_* consume; no trace edits, no MEASURED-row filtering.

Book-Author Sybil at N=10, $c = 1$ (Table **??**): KNDB 0.630 flat while all four TD algorithms collapse (TruthFinder 0.010, CRH 0.230, CATD 0.100, ACCU 0.060). Best TD (CRH) beaten by 40 points; worst (TruthFinder) by 62 points. Independent-value attack (each adversarial identity picks a distinct wrong value, no coordination): TD is flat across N=1…10, and the KNDB win narrows to 5–10 points, because independent adversaries get no trust bootstrap.

6.8 Zheng Sybil sweep and density saturation

Table **??** shows a broader Sybil sweep on Zheng d_sentiment, including N=20 (density saturation, Sybil count equals total honest votes per slot). KNDB stays flat at 0.927. CRH holds at 0.951 through N=10 and then drops to 0.000 at N=20 (a sharp cliff, not a gradual degradation). CATD holds at 0.948 through N=5, drops to 0.000 at N=10, stays at 0.000 at N=20. ACCU peaks at 1.000 at N=5 (above KNDB), drops to 0.000 at N=10 and N=20. TruthFinder degrades gradually from 0.905 down to 0.482.

The disable-and-test on the TD side (replacing each algorithm's weight-update loop with plain majority vote) shows the mechanism is bimodal, not uniformly amplifying. At sub-saturation N=1…5, CRH, CATD, and ACCU each beat plain MV by 4–26 percentage points (their log-ratio, confidence bound, or MAP identifies wrong Sybils correctly). At N=10, CATD and ACCU flip and score below MV by 29 points (the mechanism inverts: agreement now amplifies rather than defends). At N=20 all three collapse to the MV floor. TruthFinder is the only algorithm that amplifies at low N. This bimodality is the "TD is not uniformly bad" finding that shapes the Related Work discussion.

6.9 Integrity column

Table 6 reports the integrity axis on the Stage-2 YCSB grid. The grid runs three kind mixes, two contention levels (theta), and two concurrencies (c=8, c=32) against seven systems, for 78 cells total: six systems (KNDB, pg_heap, pg_conf, pg_lww, pg_mv, pg_trigger) at $3\times2\times2 = 12$ cells each, and pg_llm capped at 6 cells because the calibrated 1120 ms per-conflict latency makes higher-concurrency pg_llm runs impractical. A cell is INTEGRITY FAIL if any (entity, attribute) slot ended the trace with more than one live row. KNDB is the only system that passes on every cell.

Every trigger-based baseline fails integrity at 32 clients on the hotter theta values, because two concurrent writers each find "no incumbent" via SELECT FOR UPDATE on an empty result set, both commit, and two live rows land. The failure is not unique to pg_lww; pg_conf, pg_trigger, pg_mv, and pg_llm all exhibit it at high enough contention. KNDB's F6 advisory lock is what closes this window, and scripts/rc_invariant.sh confirms the closure by adversarial rebuild.

6.10 Real LLM calibration and non-determinism

The mock LLM in the Stage-2 baseline is calibrated to a real Anthropic Claude Haiku 4.5 measurement (bench/results/stage3_llm_calibration.jsonl, 200 real API calls, zero errors, model claude-haiku-4-5 [1]). Correctness 92.5% (185 of 200); latency mean 1120 ms, p50 940 ms, p95 1934 ms, p99 2312 ms, max 5597 ms. The mock's earlier assumed correctness of 0.65 and latency of 300 ms were both wrong; parameters were updated to match the measurement.

A separate non-determinism probe (bench/results/stage3_llm_nondeterminism_raw.jsonl, 500 real API calls: 50 conflicts × 10 replays each) measured a flip rate of 0 of 50 conflicts (all unanimous over the 10 replays) with 2 of 50 unanimous-but-wrong. Both wrong conflicts had the same signature: incumbent INFERRED with specificity 0 and confidence 0.5, candidate INFERRED with high specificity ($\gg 0$) and confidence below 0.5. KNDB's lattice picks the candidate (higher specificity within the same kind). The LLM picks the incumbent every time: it treats "same kind, higher confidence" as decisive over "same kind, higher specificity," which is the opposite of the lattice's (spec, conf) precedence order. This is a systematic disagreement about lattice ordering, not random noise; the "just run the LLM three times and vote" fix does not help.

6.11 Bypass table

Table 7 summarises the F2, F18, and F20 bypass-survival results. Each cell corresponds to a scenario in scripts/bypass.sh, scripts/update_forgery.sh, or scripts/delete_forgery.sh, which assert against both an epistemic table (fact_native) and a plain-heap table with a BEFORE-INSERT/UPDATE/DELETE trigger (fact_trigger). The adversarial disable-and-test rebuilds the extension with the corresponding TAM callback wiring line commented out and confirms the bypass returns; on F20 the dylib SHA-256 flips c873ddc4... → 1fb0d572... → c873ddc4.... The trigger column marks a scenario *bypassed* whenever a writer with INSERT+ALTER TABLE on the target can turn the trigger off;

*bypassed** marks the two rows where the trigger-based enforcer would survive if the operator opts in to ENABLE ALWAYS TRIGGER but still cannot survive DISABLE TRIGGER ALL.

6.12 Batch-size ceiling under COPY

scripts/bypass.sh sweeps $N \in \{100, 1000, 10000, 20000\}$ via real \copy at max_locks_per_transaction=64 against an epistemic table. $N = 100$, $N = 1000$, and $N = 10000$ succeed; $N = 20000$ raises ERROR 53200 out of shared memory at LockAcquireExtended, lock.c:1080. With the F18 multi_insert override disabled (rebuild), the same $N = 20000$ COPY succeeds, because heap_multi_insert takes no advisory locks; the ceiling is a direct consequence of routing through per-row enforcement, not an artefact of PostgreSQL machinery. Raising the GUC to 1024 pushes the ceiling to ~250 000 rows.

6.13 Contention control (YCSB, F9 gate)

Stage-1 gate results on YCSB-A at RC, 8 clients (bench/results/summary/gate.csv): KNDB epistemic 2057–2200 tps median (theta 0.0, 0.5), abort rate 0.050 and 0.092 with NEW_LOSES the only abort family (no 40001 seen in the gate window). pg_heap runs faster at 6600–7100 tps (no arbitration, no locks) but with no correctness guarantee. Stage-2 correctness cells (Section 6) show that raw pg_heap throughput comes with tens of thousands of duplicate live rows per 20-second window; correct-goodput (tps × (1 − abort rate) × correctness) is 0 for pg_heap on every Stage-2 cell.

6.14 Temporal-shaped benchmarks (honest zero)

KNDB scores 0.000 at $c = 1$ on MemoryAgentBench Conflict_Resolution, LongMemEval knowledge-update pairs, and MQuAKE-CF-3k edit chains (bench/results/summary/stage3.md). The reason is straightforward: these benchmarks expect last-writer-wins, and KNDB's F8 tiebreak is deliberately first-committer-wins. Both writes in each pair carry (MEASURED, spec=0, conf=1.0); the lattice cannot differentiate them by rank, so the tiebreak is what decides, and it is the wrong tiebreak for these workloads. This is disclosed in Section 8. On the same workloads pg_lww scores 1.000 at $c = 1$ and drops to 0.11–0.27 at $c = 8$; the LWW "win" at $c = 1$ is a determinism artefact of single-writer arrival order that vanishes under any contention.

7 RELATED WORK

Truth discovery. The four TD algorithms in the evaluation cover the field's canonical designs. TruthFinder [12] is the seminal fixed-point on (source trust, fact confidence). CRH [4] formulates truth discovery as joint minimisation with a log-ratio weight update that is uniquely competitive on Zheng d_sentiment below saturation, as our evaluation shows. CATD [3] adds a confidence-interval bound via $\chi^2_{\alpha/2,n}$ that is well-suited to long-tail source distributions. ACCU [2] is Bayesian MAP over per-source accuracy and, in variants with copy detection, was originally designed to address adversarial data. We use base ACCU (no copy detection) and note in Section 8 that copy-detection variants may shift the threshold. The Zheng et al. VLDB 2017 survey [13] is the source of the d_sentiment dataset and the reproducibility baseline we validated the reimplementations against. The Sybil-collapse framing draws on the same monotonicity structure the field has understood for a decade; the paper contribution is not the framing but the engine-level orthogonality that removes the collapse threshold entirely (Section 5).

Provenance in PostgreSQL. ProvSQL [8] is the mature PostgreSQL extension for semiring provenance and probabilistic queries, tracked at row-set granularity via rewriting rather than in-storage. A recent PW25 demonstration [11] adds update provenance through temporal databases with support for time-travel and undo. KNDB and ProvSQL do not overlap: ProvSQL propagates provenance through query evaluation; KNDB decides survivor selection at write time using a kind-primary total order. The two mechanisms are orthogonal and, in principle, composable.

Agent-memory and epistemic warrant. A recent line of position papers has argued that LLM agent memory is not, in practice, a database (in the sense of write-time correctness) and that the epistemic warrant conferred by an agent's tool boundary is thinner than commonly assumed. Orogat and Mansour [5] argue for rethinking data foundations for long-term AI agent memory. Romanchuk and Bondar [7] argue that tool boundaries alone do not confer epistemic warrant.

TOKI [10] is our closest sibling work and warrants direct comparison. TOKI is a theory-first bitemporal operator algebra layered over an unmodified database engine: contradictions between competing memory assertions are resolved by projecting each assertion onto valid-time and system-time axes and picking the temporally-consistent survivor. TOKI has no epistemic-kind axis; every assertion is treated as a value carrying only temporal metadata. TOKI explicitly defers threat-model integration to future work. KNDB's contribution is precisely that deferred piece plus an engine-level realisation: an orthogonal kind axis that is assigned by the storage engine at write time and therefore cannot be forged by an untrustworthy writer, implemented as a PostgreSQL table access method rather than as an operator layered above one. TOKI's temporal semantics and KNDB's epistemic kind axis are compatible along different axes and, in principle, composable.

Serialisable snapshot isolation. KNDB's concurrency behaviour under SERIALIZABLE is standard PostgreSQL SSI [6]. The AM does not add a slot-level SIRead lock; predicate locking is inherited from heapam. Our early proof-of-concept had wrapped PredicateLockTID on a synthetic slot-hash TID, and F1 audited that wrapper and found it decorative (the wrapper's supposed contribution was already provided by heap's PredicateLockRelation plus CheckForSerializableConflictIn). The wrapper was removed.

PostgreSQL as a research substrate. We build on the PostgreSQL 18 table access method interface [9]; the API surface has been stable across three major versions and is a productive vehicle for storage-layer research that must interoperate with a real query planner, executor, WAL, and recovery pipeline. Because rows on disk are plain heap tuples, existing PostgreSQL tools (pg_dump, pg_upgrade, logical replication, pg_basebackup) work on KNDB tables without change.

8 LIMITATIONS

We are explicit about where KNDB does not win.

Truth discovery wins sub-saturation. On Zheng d_sentiment below the density-saturation cell (N < 20), CRH matches or beats KNDB on the confidence-forgery workload, and ACCU peaks above KNDB at N=5 (1.000 versus 0.927). CRH's log-ratio, CATD's χ^2 confidence bound, and ACCU's Bayesian MAP each identify perfectly-wrong Sybils correctly when the coalition is small relative to per-slot honest votes. The paper's contribution is not that KNDB beats every TD baseline on every cell; it is that KNDB is the only system whose k^* is unbounded, which becomes decisive at and above the saturation cell.

Zero on temporal knowledge-editing benchmarks. On MemoryAgentBench Conflict_Resolution (37 820 writes), LongMemEval knowledge-update (156 writes), and MQuAKE-CF-3k (12 030 writes), KNDB scores 0.000 at $c = 1$ because these benchmarks expect the later write to win and KNDB's tiebreak is first-committer-wins. A dedicated "epistemic table with last-writer-wins tiebreak" would be a straightforward configuration knob, but the current implementation does not expose it. Naive pg_lww scores 1.000 on these benchmarks at $c = 1$ and collapses to 0.11–0.27 at $c = 8$, so LWW is not a general answer either; the tiebreak choice is a workload-specific configuration and no fixed choice is right for all workloads.

Reputation farming defeats $k^* = \infty$. The $k^* = \infty$ argument for KNDB against Sybil coordination depends on adversarial identities appearing fresh in the write stream, so that engine-assigned kind places them in INFERRED rather than MEASURED. A determined adversary who knows the tier-mapping rule can farm the reputation signal to reach Tier A before attacking: on Zheng, complete the qualification test (question ids 2000…2019) with correct answers so quali_acc matches Tier A; on Book-Author, publish n_listings volume on benign books to enter the top-K by source volume. Once inside Tier A, the adversary's writes are MEASURED and the lattice defends nothing. This is a real boundary of the mechanism. The formal argument holds for the write-time snapshot; long-lived reputation-farming requires an external audit or a stricter tier-assignment policy (e.g., cryptographically-signed source attestations, KYC on new bookstore registrations, or an audit that revokes Tier A on posthoc discovery of adversarial behaviour) that is out of scope for this paper. The two attack strategies that KNDB *does* defend against are: (1) an adversary who has not farmed reputation and appears in the write stream in INFERRED regardless of self-reported confidence, which is the confidence-forgery workload of Section 6, and (2) an adversary who has INSERT plus ALTER TABLE on the target table but no ability to change its tier assignment, which is the write-path bypass surface of Section **??**.

R2 and R5 SPI cost per non-MEASURED insert. Both R2 (source resolution against epistemic.source_registry) and R5 (slot-kind check against epistemic.slot_kind) run a full SPI_connect / SPI_execute_with_args / SPI_finish cycle per candidate row. Both run BEFORE the F6 per-slot advisory lock is acquired (epistemic_am.c steps 1 vs 2), so neither introduces a deadlock risk against other advisory-lock holders. The per-row cost is measurable but small: a microbenchmark of 5 000 INFERRED inserts versus 5 000 MEASURED inserts on a fresh cluster puts R2's added overhead (one count(*) SPI call with a one-element text-array parameter) at ~7 microseconds per row, or roughly 35% on top of the ~19 microsecond MEASURED baseline (which itself pays R5's SPI plus the overlap scan). Backend-local caching of source_registry keyed off CacheRegisterRelcacheCallback would drive R2's per-row cost to near zero after a warm start and is a straightforward optimisation for a follow-up paper; we do not ship it here because the current cost is well below any workload's practical throughput floor and the caching adds an invalidation contract we would want to test more thoroughly than the F20 time budget allowed.

Batch-size ceiling of ~15 000 rows per transaction at default GUC. Documented in Section 4. Raising max_locks_per_transaction to 1024 (a postgresql.conf change requiring restart) pushes the ceiling to ~250 000. Applies to COPY equally, because the F18 multi_insert override routes batched COPY through the same per-row advisory lock. We chose not to drop the advisory lock in the batch path because doing so silently re-opens the F6 concurrency integrity leak.

ALTER TABLE SET ACCESS METHOD heap **out of scope.** The table owner can rewrite the relation onto plain heap, at which point the TAM callback is no longer bound to the relation and every subsequent write goes through unmodified heap. This is a schema-change threat, not a write-path threat. It is disclosed here and in the README threat-model section. Restricting the DDL is out of scope for this proof of concept; the natural next step is a PostgreSQL security policy that denies SET ACCESS METHOD away from epistemic for tables the epistemic contract is claimed over.

Custom WAL rmgr is annotation only. Recovery works byte-for-byte with the extension's WAL emitter disabled. Durability of the row comes from heap's XLOG_HEAP_INSERT. The custom rmgr 128 is retained as a named channel for a future logical decoding consumer to hook, but does not itself contribute to crash recovery. PostgreSQL's stock pg_waldump does not load custom rmgrs and renders these records as custom128 UNKNOWN. This is disclosed.

Eviction atomicity is PostgreSQL's. The three-step eviction (winner insert, incumbent sys_time close, audit row) executes inside one top-level PostgreSQL transaction. Atomicity is inherited, not novel. What the TAM contributes is that the three steps are inside the tuple_insert callback and cannot be forgotten by a user-space writer, which is a convenience property, not a novel durability property.

Seven of 42 TAM callbacks overridden. We override tuple_insert, multi_insert, tuple_update (F20), tuple_delete (F20), tuple_insert_speculative (F21), tuple_complete_speculative (F21), and relation_toast_am. Every other callback (scan, index build, VACUUM, analyse, replication copy, cluster, tuple lock, ttids, and so on) is heap's, unmodified. Table 1 in Section 4.9 classifies all 42 callbacks with

citable reasons. We inherit heap's behaviour on the 35 non-overridden ones, including its bugs. This is deliberate; the AM is a write-time enforcement point, not a storage-format replacement.

Serialisable-level guarantees are PostgreSQL's. Under SERIALIZABLE, KNDB inherits heap's predicate locking (PredicateLockRelation) rather than a slot-level SIRead lock. Two writers on non-overlapping slots may still hit an SSI abort because the predicate lock granulates to relation-level. This is a known SSI property in PostgreSQL [6]; adding slot-level predicate locking would require modifying predicate.c (a new lock target type) and is out of scope.

9 CONCLUSION

KNDB reports an engineering result: seven overrides in the PostgreSQL 18 table access method interface, an engine-assigned epistemic kind column, a per-slot advisory transaction lock, and a first-committer-wins tiebreak on true precedence ties are sufficient to make PostgreSQL survive a confidence-forgery attack that flattens every conventional PostgreSQL baseline. On two workloads with entirely different provenance shapes (source-tier data fusion, per-worker crowdsourcing qualification), the win is 63 points and 92.7 points over a confidence-only arbitrator and is proven load-bearing on the kind axis by source-rebuild disable-and-test. Against classic truth-discovery baselines the picture is more nuanced: KNDB is competitive below the per-dataset density-saturation cell and dominant at and above it, because kind rank is engine-assigned from independent metadata that adversarial identities do not possess. The threshold theorem $k^* \approx \rho_{\text{alg}} \cdot h_{\text{top}}$ makes the Sybil-collapse cell predictable from per-slot honest-support alone. What KNDB does not do is win on workloads whose ground truth is last-writer-wins; we are explicit about that, and about the batch-size ceiling, the schema-change threat, the annotation-only WAL role, and the reputation-farming boundary of the $k^* = \infty$ argument, throughout the paper. The value of this work, in our view, is the demonstration that an epistemic-kind primary sort key is deployable in a mainstream OLTP database in roughly 2400 lines of new C at the TAM layer, no core patch required, with a completeness argument over the full 42-callback TAM interface (Table 1).

ACKNOWLEDGMENTS

The authors used a large language model as a coding assistant and evaluation harness developer during this work. All system-level design, empirical results, formal arguments, and paper prose were reviewed and validated by the authors.

REFERENCES

[1] Anthropic. 2025. Claude Haiku 4.5. https://www.anthropic.com/claude/haiku.
[2] Xin Luna Dong, Laure Berti-Equille, and Divesh Srivastava. 2009. Integrating Conflicting Data: The Role of Source Dependence. *Proc. VLDB Endow.* 2, 1 (2009), 550–561. https://doi.org/10.14778/1687627.1687690
[3] Qi Li, Yaliang Li, Jing Gao, Lu Su, Bo Zhao, Murat Demirbas, Wei Fan, and Jiawei Han. 2014. A Confidence-Aware Approach for Truth Discovery on Long-Tail Data. *Proc. VLDB Endow.* 8, 4 (2014), 425–436. https://doi.org/10.14778/2735496.2735505
[4] Qi Li, Yaliang Li, Jing Gao, Bo Zhao, Wei Fan, and Jiawei Han. 2014. Resolving Conflicts in Heterogeneous Data by Truth Discovery and Source Reliability Estimation. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 1187–1198. https://doi.org/10.1145/2588555.2610509
[5] Abdelghny Orogat and Essam Mansour. 2026. Is Agent Memory a Database? Rethinking Data Foundations for Long-Term AI Agent Memory. arXiv preprint. arXiv:2605.26252
[6] Dan R. K. Ports and Kevin Grittner. 2012. Serializable Snapshot Isolation in PostgreSQL. *Proc. VLDB Endow.* 5, 12 (2012), 1850–1861.
[7] Oleg Romanchuk and Roman Bondar. 2026. Semantic Laundering in AI Agent Architectures: Why Tool Boundaries Do Not Confer Epistemic Warrant. arXiv preprint. arXiv:2601.08333
[8] Pierre Senellart, Louis Jachiet, Silviu Maniu, and Yann Ramusat. 2018. ProvSQL: Provenance and Probability Management in PostgreSQL. *Proc. VLDB Endow.* 11, 12 (2018), 2034–2037. https://doi.org/10.14778/3229863.3236253
[9] The PostgreSQL Global Development Group. 2025. PostgreSQL 18: Table Access Method Interface. https://www.postgresql.org/docs/18/tableam.html.
[10] Ziming Wang. 2026. TOKI: A Bitemporal Operator Algebra for Contradiction Resolution in LLM-Agent Persistent Memory. arXiv preprint. arXiv:2606.06240
[11] Albert Ariel Widiaatmaja, Belkis Djeffal, Ashish Dandekar, and Pierre Senellart. 2025. Demonstration of ProvSQL Update Provenance through Temporal Databases. In *Proceedings of ProvenanceWeek 2025 (PW25)*. https://doi.org/10.1145/3736229.3736253
[12] Xiaoxin Yin, Jiawei Han, and Philip S. Yu. 2007. Truth Discovery with Multiple Conflicting Information Providers on the Web. In *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 1048–1052. https://doi.org/10.1145/1281192.1281309
[13] Yudian Zheng, Guoliang Li, Yuanbing Li, Caihua Shan, and Reynold Cheng. 2017. Truth Inference in Crowdsourcing: Is the Problem Solved? *Proc. VLDB Endow.* 10, 5 (2017), 541–552. https://doi.org/10.14778/3055540.3055547

Table 1: Exhaustive audit of the PG 18 TableAmRoutine interface (access/tableam.h, REL_18_STABLE, 42 callbacks). The seven OVERRIDDEN rows are load-bearing: each has a source-rebuild disable-and-test in scripts/ that proves the bypass returns when the wiring is commented out (see DECISIONS.md F9, F18, F20, F21). The other 35 rows are inherited from heap; the "why safe" column cites the specific reason we do not need to intervene.

callback	class	tableam.h	why safe / role
Slot callbacks			
slot_callbacks	read-only	302	Returns the TupleTableSlotOps for tuples of this AM. Pure factory; no state mutation.
Table scan callbacks			
scan_begin	read-only	326–330	Starts a read-only scan with a snapshot. No mutation path.
scan_end	read-only	336	Releases scan resources. No mutation.
scan_rescan	read-only	342–344	Restarts scan with new params. No mutation.
scan_getnextslot	read-only	349–351	Returns next tuple into slot. No mutation.
scan_set_tidrange	read-only	370–372	Sets bounds for a TID-range scan. No mutation.
scan_getnextslot_tidrange	read-only	378–380	TID-range variant of scan_getnextslot. No mutation.
Parallel scan			
parallelscan_estimate	read-only	391	Sizes shared-mem DSM. No mutation.
parallelscan_initialize	read-only	398–399	Initialises a ParallelTableScanDesc. No user-visible state.
parallelscan_reinitialize	read-only	405–406	Re-initialises the DSM for a new scan. No mutation.
Index scan			
index_fetch_begin	read-only	422	Opens an index-scan fetch state. No mutation.
index_fetch_reset	read-only	428	Resets between index scans. No mutation.
index_fetch_end	read-only	433	Releases index-scan state. No mutation.
index_fetch_tuple	read-only	455–459	Fetches by TID after visibility test. No mutation.
Non-modifying tuple ops			
tuple_fetch_row_version	read-only	472–475	Fetches a tuple version. No mutation.
tuple_tid_valid	read-only	480–481	Predicate on tid. No mutation.
tuple_get_latest_tid	read-only	487–488	Walks the update chain to the newest visible tid. Reads only.
tuple_satisfies_snapshot	read-only	494–496	Visibility test. No mutation.
index_delete_tuples	delegated-storage	499–500	Bottom-up index-delete driver. In heap's implementation, no user-visible tuples are removed; dead line pointers are reclaimed. Our epistemic invariants live in *live* rows (sys_time @> now()); dead-line-pointer reclamation cannot affect them.
Mutating tuple ops			
tuple_insert	**OVERRIDDEN**	508–511	F1–F16: R1–R5, F6 advisory lock, precedence + F8 xmin tiebreak, eviction audit, sys_time close, rmgr-128 marker.
tuple_insert_speculative	**OVERRIDDEN**	513–519	**F21**: R1–R5, advisory lock, precedence; heap does the speculative write; deferred eviction stashed for confirm.
tuple_complete_speculative	**OVERRIDDEN**	521–525	**F21**: on succeeded=true, drain the pending eviction (audit + sys_time close + rmgr-128); on succeeded=false, discard.
multi_insert	**OVERRIDDEN**	527–529	F18: fan out to epistemic_tuple_insert_impl per slot; closes the COPY CIM_MULTI bypass F9 identified.
tuple_delete	**OVERRIDDEN**	531–539	F20: refuse DELETE outright with ERRCODE_FEATURE_NOT_SUPPORTED.
tuple_update	**OVERRIDDEN**	541–551	F20: refuse any UPDATE whose new (ep_kind, ep_specificity, ep_confidence) differs from the incumbent's.
tuple_lock	delegated-storage	553–562	Row-level lock acquisition (SELECT FOR UPDATE, replica-apply conflict resolution). Sets xmax to lock the tuple but does not mutate user columns; any subsequent modification routes back through tuple_update or tuple_delete, both overridden.
finish_bulk_insert	delegated-storage	576	Optional per-bulk-insert flush. In-tree AMs no longer use it; heap's binding is NULL (heapam_handler.c:2661). Any prior mutation went through tuple_insert or multi_insert.
DDL			
relation_set_new_filelocator	delegated-storage	600–604	Allocates a new physical file for TRUNCATE/CLUSTER/REINDEX-style rewrites (heapam_handler.c:583–622: RelationCreateStorage + optional init-fork). Storage is empty on return; any subsequent rewrite repopulates via tuple_insert or multi_insert.
relation_nontransactional_truncate	delegated-storage	614	Truncates the file to zero size (heap: RelationTruncate, no WAL). Removes all rows including epistemic incumbents. This IS a policy-relevant bypass surface for the eviction audit — but it is TRUNCATE DDL, guarded by TRUNCATE privilege, and out of scope for the write-path threat (Section 2.3). Flagged as a schema-change bypass alongside ALTER TABLE ...SET ACCESS METHOD heap (Section 9).
relation_copy_data	delegated-storage	622–623	Copies the physical file to a new tablespace (ALTER TABLE ...SET TABLESPACE). Rows are copied byte-for-byte; no epistemic-level policy applies.
relation_copy_for_cluster	delegated-storage	626–635	Copies rows during CLUSTER / VACUUM FULL. Heap re-inserts via raw_heap_insert, not table_tuple_insert, so the epistemic checks do NOT re-fire during the rewrite. Accepted because CLUSTER/VACUUM FULL is a DDL-privileged operation that preserves the current visible rows; it cannot introduce a forgery, only reorder existing rows. Flagged as an in-scope limitation on par with the schema-change threat.
relation_vacuum	delegated-storage	652–654	Standard VACUUM (dead-tuple reclamation, freeze). Cannot resurrect a dead row or invent a live one; the "live row per slot" invariant is preserved.
scan_analyze_next_block	read-only	673–674	ANALYZE block sampling. Read-only.
scan_analyze_next_tuple	read-only	684–688	ANALYZE tuple sampling. Read-only.
index_build_range_scan	read-only	691–701	Feeds tuples to IndexBuildCallback during index build. Read-only from the AM's perspective; the index AM writes to its own relation.
index_validate_scan	read-only	704–708	Concurrent-index-build validation phase. Read-only against the table.
Misc			
relation_size	read-only	724	Returns bytes for a fork. Pure metadata.
relation_needs_toast_table	delegated-decide	734	Heap's decision, whether a TOAST table is needed. We accept it.
relation_toast_am	**OVERRIDDEN**	741	Return HEAP_TABLE_AM_OID so the TOAST table is plain heap; otherwise PG's index build for the TOAST table trips heap_getnext's rd_tableam == GetHeapamTableAmRoutine() identity check (heapam.c:1352).
relation_fetch_toast_slice	read-only	748–752	Fetches a slice of a TOAST'd value. Read-only.
Planner			
relation_estimate_size	read-only	770–772	Row-count / page-count estimate for the planner. Pure.
Executor			
scan_bitmap_next_tuple	read-only	792–796	Fetch next visible tuple from a bitmap scan. Read-only.
scan_sample_next_block	read-only	823–824	Sample-scan block selection. Read-only.
scan_sample_next_tuple	read-only	839–841	Sample-scan tuple selection. Read-only.

Table 6: Integrity FAIL counts on the Stage-2 grid (12 cells per system; pg_llm at 6). From bench/results/summary/stage2.md F14 addendum.

system	INTEGRITY FAIL cells
KNDB epistemic	0 / 12
pg_conf	12 / 12
pg_heap	12 / 12
pg_lww	10 / 12
pg_trigger	10 / 12
pg_mv	9 / 12
pg_llm	3 / 6

Table 7: Bypass scenarios and outcomes. From scripts/bypass.sh, scripts/update_forgery.sh, scripts/delete_forgery.sh transcripts and README threat-model enumeration.

scenario	KNDB	trigger
DISABLE TRIGGER ALL	blocked	bypassed
session_replication_role	blocked	bypassed*
COPY single-row (CIM_SINGLE)	blocked	blocked
COPY batch (CIM_MULTI, post-F18)	blocked	bypassed
INSERT, INSERT SELECT	blocked	blocked
UPDATE of prefix (post-F20)	blocked	bypassed
DELETE incumbent (post-F20)	blocked	bypassed
ON CONFLICT DO UPDATE (post-F21)	blocked†	n/a
ON CONFLICT DO NOTHING (post-F21)	blocked†	n/a
Logical replication apply	blocked	bypassed*
Out of scope:		
SET ACCESS METHOD heap	schema-change threat	
TRUNCATE, CLUSTER, VACUUM FULL	DDL-privileged; see Table 1	

† Blocked by the F21 override; but the SQL surface for this attack is not currently reachable because unique/exclusion constraint creation on epistemic tables fails at heap_getnext's rd_tableam identity check (heapam.c:1352), so ON CONFLICT (col) has no arbiter to bind. Load-bearing is proven via the C-level probe epistemic._probe_speculative_insert and the source-rebuild disable-and-test in Section 6.6. The overrides are defensive: the engine-in-storage claim must not depend on that accidental index-support gap persisting.